4. Hafta

React I: Bileşenler

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- React'in bildirimsel programlama modelini Hafta 3'ün elle DOM güncelleyen kodundan farkını açıklayarak, Vite'ın `react-ts` şablonuyla bir React+TypeScript projesi kurup TSX söz diziminde (ifade gömme, `className`, tek kök öğe/Fragment) basit bir bileşen yazabileceksiniz;
- `props` ve TypeScript arayüzleriyle (`interface XProps`) tiplenmiş, `children` kabul edebilen fonksiyon bileşenleri oluşturup bileşen ağacı içinde tek yönlü veri akışını (props yalnızca yukarıdan aşağıya akar) açıklayabileceksiniz;
- `useState` ile bir bileşenin kendi durumunu tutmasını, değişmezlik ilkesine uyarak (nesne/dizi doğrudan değiştirmek yerine kopyalayarak/yeni değer üreterek) durumu güncellemeyi ve tiplenmiş olay işleyicileriyle (`onClick` vb.) kullanıcı etkileşimine tepki vermeyi uygulayabileceksiniz;
- `map` ile bir diziden bileşen listesi üretip her öğeye doğru `key` atayabilecek, koşullu render kalıplarını (`&&`, üçlü operatör, erken dönüş) kullanarak boş liste gibi durumları yönetebileceksiniz;
- Bu becerileri birleştirip Hafta 3'ün elle yazılmış DOM kodunu (`EtkinlikKarti`, `EtkinlikListesi`, `Koltuk`, `KoltukPlani`) React bileşenlerine dönüştürerek Etkinlik/Bilet Sistemine ilk React katmanını ekleyebileceksiniz.

Hafta 3'te bir koltuğa tıklandığında hem koltuğun kendi rengini hem de sayfanın başka bir yerindeki "seçim özeti" metnini ayrı ayrı, elle güncellemeniz gerekiyordu — iki güncellemeden birini unutmak ekranı gerçek durumdan farklı gösterebilirdi. Bu hafta, Hafta 3'te kurduğunuz Vite/npm/TypeScript araç zincirinin üzerine **React** eklenir; React'i bu haftaya kadar hiç görmediniz, ama Hafta 3'te TypeScript'in tip sistemini ve DOM'u elle yönetmenin ne kadar kırılgan olduğunu bizzat yaşadınız — bu hafta bu deneyimin üzerine "aynı işi neden/nasıl daha az elle" sorusunun cevabı olarak React inşa edilir. Bu hafta React yalnızca sabit (mock) veriyle çalışır — `fetch`, gerçek bir API, formlar, `useEffect` ve durum kaldırma Hf.5-7'nin konusudur. Dört ünite boyunca sırasıyla React'in bildirimsel modelini ve TSX söz dizimini (Ünite 1), `props`/`children` ile tiplenmiş bileşenleri (Ünite 2), `useState` ile durumu ve olay işleyicilerini (Ünite 3) ve son olarak liste/koşullu render kalıplarını

(Ünite 4) göreceksiniz — son ünitenin sonunda Etkinlik/Bilet Sisteminin ön yüzü artık tamamen React bileşenlerinden oluşacaktır.

4.1 Neden Bileşen? React'e Giriş ve TSX

1 • Ön-öğretim — Neden Önemli?

Hafta 3'te bir koltuğa tıklandığında hem koltuğun kendi rengini hem de sayfanın başka bir yerindeki "seçili koltuk" metnini ayrı ayrı, elle güncellemeniz gerekiyordu. Bu iki güncellemeden birini unutsaydınız ekran, gerçek durumdan farklı bir şey gösterirdi — kullanıcı "seçili" sandığı bir koltuğun aslında seçili görünmediğini fark ederdi. Bu ünite, "durumu tanımlayın, görünümü kendisi güncellesin" fikrini tanıtarak bu senkronizasyon yükünü ortadan kaldırır.

1 • Ön-öğretim — Ön Bilgi

Hafta 3 Ünite 4'ün `koltukPlani_uret`/`koltuk_tiklama` fonksiyonlarını hatırlayın — `querySelector`/`addEventListener`/ `classList` ile DOM'u elle yönettiğiniz, o haftanın son bölümünde "React'e geçiş gerekçesi" olarak önizlenen kodlar. Hafta 3 Ünite 2'nin Vite/npm bilgisini de hatırlayın — bu hafta yalnızca proje şablonu değişir (`vanilla-ts` yerine `react-ts`), Vite'ın kendisi ve `npm run dev`/`npm run build` ayrımı aynı kalır.

Hafta 3'ün sonunda, Hafta 2'nin statik sayfasına *davranış* eklediniz — ama bunu yaparken her koltuğun rengini ve sayfanın başka bir köşesindeki seçim özetini ayrı ayrı, elle güncellemeniz gerekti. Bu ünite bu elle güncelleme yükünün nereden geldiğini açıklar ve React'in bu yükü nasıl kaldırdığını, hangi araçlarla ve hangi söz dizimiyle (TSX) çalıştığını tanıtır.

4.1.1 Zorunlu koddan bildirimsel modele

Hafta 3 Ünite 4'te bir koltuğa tıklandığında şu adımları siz, elle yazdınız: önce `querySelector` ile ilgili `button` öğesini bulun, sonra `classList.toggle` ile sınıfını değiştirin, sonra ayrı bir fonksiyonla seçim özetinin metnini güncelleyin. Bu yaklaşımın adı **zorunlu programlama**dır: DOM'u nasıl güncelleyeceğinizi — hangi öğeyi bulacağınızı, hangi sırayla hangi özelliği değiştireceğinizi — baştan sona siz yazarsınız. React ise tam tersi bir modelle çalışır: yalnızca "durum şu anda böyleyse görünüm böyle olmalı" tanımını yazarsınız, güncellemenin gerçekten nasıl yapılacağına React karar verir. Buna **bildirimsel programlama** (*declarative programming*) denir.

zorunlu programlama: "nasıl yapılacağını" adım adım kodla belirten yaklaşım (*imperative programming*).

Analoji

Bu ayrımı bir restoranda sipariş vermekle mutfağa girip yemeği kendiniz pişirmeye benzetin: zorunlu yaklaşımda mutfağa girip her adımı (ateşi yakın, tavayı koyun, malzemeyi ekleyin) siz yaparsınız; bildirimsel yaklaşımda garsona yalnızca ne istediğinizi (siparişi) söylersiniz — bunun nasıl pişirileceğine mutfak (React) karar verir. Siz "bu koltuk şimdi seçili olsun" dersiniz, ekranı hangi adımlarla güncelleyeceğine React karar verir.

Şekil 4.1 bu iki modeli yan yana karşılaştırır: solda Hafta 3'ün "siz bulun, siz değiştirin" akışı, sağda React'in "yalnızca durumu bildirin, React günceller" akışı görünür.



Şekil 4.1: Zorunlu (Hafta 3, elle DOM güncelleme) ile bildirimsel (React, durum → görünüm) programlama modellerinin karşılaştırması.

Bu noktada **React nedir** sorusunun net cevabı şudur: React, kullanıcı arayüzü oluşturmak için kullanılan bir kütüphanedir; bir bileşenin "şu an durum böyleyse görünüm böyle olsun" tanımını alır, durum her değiştiğinde gerçek DOM'u — hangi öğenin hangi sırayla değişeceğine sizin yerinize karar vererek — kendisi günceller.

4.1.2 Sanal DOM ve uzlaştırma: React perde arkasında ne yapar?

Sanal DOM: React'in her güncellemede önce ürettiği, hafif ve gerçek tarayıcı DOM'una henüz uygulanmamış taslak ağaç (*virtual DOM*).

React'in "durumu bildirin, gerisini bana bırakın" sözü kulağa sihir gibi gelebilir; aslında iki adımlı, anlaşılır bir mekanizmaya dayanır. Her güncellemede React önce hafif, bellekte tutulan bir taslak ağaç — **Sanal DOM** — hesaplar. Ardından bu yeni taslağı bir önceki taslakla karşılaştırır; bu karşılaştırma sürecine **uzlaştırma** (*reconciliation*) denir. Uzlaştırmanın sonunda React yalnızca değişen kısmı gerçek tarayıcı DOM'una uygular — sayfanın tamamını yeniden çizmez.

Analoji

Sanal DOM ve uzlaştırmayı bir taslak üzerinde düzeltme yapıp yalnızca değişen satırları temize çekmeye benzetin: her seferinde tüm belgeyi baştan yazmak yerine, önce bir taslak (Sanal DOM) hazırlanır, eski taslakla karşılaştırılır, yalnızca farklı kısımlar temiz kâğıda (gerçek DOM'a) geçirilir. Bu sayede binlerce satırlık bir belgede tek bir kelime değişse bile belgenin tamamı yeniden yazılmaz.

Bu ünitede Sanal DOM ve uzlaştırma yalnızca fikir düzeyinde kalsın — Ünite 3'te bir `useState` çağrısının bu süreci nasıl tetiklediğini, Ünite 4'te ise `key`'in uzlaştırmayı nasıl doğru yönlendirdiğini somut örneklerle göreceksiniz.

4.1.3 Vite `react-ts` şablonuyla proje kurulumu

Hafta 3'te `npm create vite@latest` komutunu `vanilla-ts` şablonuyla çalıştırmıştınız. Bu hafta tek fark, şablon adıdır:

```
Terminal
$ npm create vite@latest istemci -- --template react-ts
$ cd istemci
$ npm install
$ npm run dev
```

Oluşan klasör yapısı Hafta 3'ün `vanilla-ts` projesine şaşırtıcı derecede benzer: yine bir `index.html`, yine bir `src/main.tsx` (Hafta 3'teki `src/main.ts`'nin karşılığı, uzantı artık `.tsx`), yine bir `tsconfig.json` ve `package.json` vardır. Aradaki fark, `package.json` içindeki bağımlılıklardır — artık `react` ve `react-dom` da listelenir:

```
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  },
  "devDependencies": {
    "@types/node": "^24.13.3",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "typescript": "^7.0.2",
    "vite": "^8.2.2"
  }
```

İkinci fark, `vite.config.ts` dosyasına eklenen bir eklentidir. Bu eklenti, Vite'a TSX/JSX söz dizimini derlerken React'in kendine özgü dönüşümünü (JSX'i fonksiyon çağrılarına çevirme) uygulamasını söyler:

```
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vite';

export default defineConfig({
  plugins: [react()],
});
```

`index.html` dosyası da Hafta 3'tekiyle neredeyse birebir aynıdır — yine boş bir `<div id="root">` kökü, yine bunu başlatan bir `<script type="module">` etiketi:

```
<div id="root"></div>
<script type="module" src="/src/main.tsx"></script>
```

`src/main.tsx` dosyasının kendi içeriğini Ünite 4'te, tüm bileşenler hazır olduğunda ayrıntılı göreceksiniz — şimdilik bu dosyanın görevinin, bileşen ağacınızın kökünü bu `root` öğesine bağlamak olduğunu bilmeniz yeterlidir.

4.1.4 TSX sözdizimi

Hafta 3'ün `.ts` dosyalarında hiç HTML'e benzer bir yazım görmediniz; React bileşenleri ise genellikle `.tsx` uzantılı dosyalarda, HTML'e benzeyen bir söz dizimiyle yazılır. Bu söz dizimine **JSX / TSX** denir — JavaScript dünyasında `.jsx`, TypeScript dünyasında `.tsx` uzantısı kullanılır. Kritik ayrıntı şudur: `<h1>Merhaba</h1>` gibi bir ifade tarayıcının anladığı gerçek HTML değildir; Vite'ın eklediğiniz eklenti sayesinde derleme sırasında bunu bir JavaScript fonksiyon çağrısına çevirir. Bu yüzden TSX içine süslü parantezle bir JavaScript/TypeScript **ifade gömmesi** (*expression embedding*) yapabilirsiniz: `{degisken}` ya da `{1 + 1}` gibi, yalnızca bir değer üreten ifadeler — `if`/`for` gibi bir deyim süslü parantez içine yazılamaz, çünkü bunlar bir değer üretmez.

JSX / TSX: HTML'e benzeyen ama aslında JavaScript/TypeScript fonksiyon çağrılarına derlenen bir söz dizimi (*JSX/TSX*).

TSX'in HTML'den farklı davrandığı bir nokta daha vardır: `class` özniteliği yerine `className` yazılır. Bunun nedeni basittir — `class`, JavaScript dilinde zaten ayrılmış bir sözcüktür (bir sınıf tanımlamak için kullanılır), bu yüzden TSX bu çakışmayı önlemek için farklı bir isim seçer.

İpucu

Editörünüz `class="..."` yazdığınızda genellikle bunu kırmızı alt çizgiyle işaretler ya da otomatik olarak `className`'e çevirir — bu uyarıyı görürseniz TSX dosyasında olduğunuzu ve `className` yazmanız gerektiğini hatırlayın.

Son olarak, bir React bileşeninin döndürdüğü JSX'in tek bir kök öğesi olmalıdır — iki kardeş öğeyi doğrudan art arda döndüremezsiniz. Birden fazla kardeş öğe döndürmeniz gerekiyorsa ve bunun için ekstra bir `<div>` eklemek istemiyorsanız, görünmez bir sarmalayıcı olan **Fragment** kullanabilirsiniz — kısa yazımı `<>...</>`'dir; ekrana hiçbir ek etiket basmaz.

Fragment: Birden fazla kardeş öğeyi ekstra bir DOM düğümü oluşturmadan gruplamaya yarayan görünmez sarmalayıcı (*Fragment*).

Ortak Hata / Tuzak

Yeni başlayanlar sıklıkla bir bileşenden iki kardeş öğeyi (`<h1>...</h1>` ve `<p>...</p>` gibi) doğrudan art arda döndürmeye çalışır ve `tsc`/Vite bunu bir sözdizimi hatasıyla reddeder. Çözüm ya ikisini ortak bir `<div>` ile sarmak ya da — ekstra bir DOM düğümü oluşturmak istemiyorsanız — bir Fragment (`<>...</>`) ile sarmaktır. Fragment, ekrana hiçbir görünür/DOM izi bırakmadığı için stillemeyi (CSS seçicilerini) etkilemez.

Hafta 3'ün `koltukPlani_uret` ve `koltuk_tiklama` fonksiyonları öğeleri `createElement` ile elle oluşturup `addEventListener` ile olay dinleyicisi elle bağlıyordu. Ünite 3-4'te tam olarak bu iki fonksiyonun yerini, TSX döndüren ve kendi durumunu tutan bir `Koltuk` bileşeninin nasıl aldığını göreceksiniz.

3 • Katılım — Şimdi Siz Deneyin

`npm create vite@latest -- --template react-ts` ile bir proje oluşturun, `src/App.tsx` dosyasını açın. Süslü parantez içindeki (`{...}`) bir ifadeyi (ör. bir sayacın başlangıç değerini) değiştirip kaydedin; tarayıcının ne yaptığını gözlemleyin.

Geribildirim. Dosyadaki `<h1>`/`<button>` gibi etiketler HTML'e benzer ama aslında JavaScript fonksiyon çağrılarına derlenir; süslü parantez içine yalnızca bir değer üreten ifade yazılabilir (`if`/`for` gibi bir deyim yazılamaz). Değişikliği kaydettiğinizde Vite'ın anlık güncelleme özelliği (*Hot Module Replacement*, HMR) sayfayı tamamen yeniden yüklemeden yalnız değişen kısmı günceller — tarayıcı sekmesi yenilenmez, yalnızca ilgili bileşen tazelenir.

Kendinizi Sınayın 4.1

Aşağıdaki bileşen neden `tsc`/derleme aşamasında sorun çıkarır (ya da beklenmedik biçimde davranır)? Düzeltin.

```
function Baslik() {
  return (
```

```
    <h1>Hoşgeldiniz</h1>
    <p>Etkinlik sayısı: 3</p>
  );
}
```

5 • İzleme — Aktarım — Başka Nerede?

Bu haftadan itibaren Etkinlik/Bilet Sisteminin ön yüzü artık HTML'e elle yazılan ya da DOM API'siyle elle üretilen öğeler değil, TSX bileşenleridir. Bugün kurduğunuz `react-ts` iskeleti Hf.5, Hf.7 ve sonrasında aynı `istemci/` klasöründe büyümeye devam edecektir — proje kurulumunu her hafta yeniden yapmayacaksınız, yalnızca yeni bileşenler ve yetenekler ekleyeceksiniz.

4.2 props ve Tiplenmiş Bileşen

1 • Ön-öğretim — Neden Önemli?

Hafta 3'te bir etkinlik kartını DOM'a elle basarken, kartın HTML yapısı tek bir yerde tanımlıydı ama her etkinlik için bu kod bir döngüde tekrar tekrar çalıştırılıyordu — kartın kendisi gerçek, tekrar kullanılabilir bir "parça" değildi, yalnızca bir döngünün her turunda yeniden üretilen bir HTML metniydi. Bu ünite bir kartı gerçek bir yeniden kullanılabilir birim — bir bileşen — olarak tanımlamayı öğretir.

1 • Ön-öğretim — Ön Bilgi

Hafta 3 Ünite 3'ün `Etkinlik` arayüzünü hatırlayın — bu hafta aynen, hiç değiştirilmeden kullanılacak. Ayrıca YG I/II'den tanıdık bir alışkanlığı hatırlayın: bir fonksiyona parametre geçirmek, ör. C++'ta `const Etkinlik&` ile bir fonksiyona "salt okunur" biçimde veri geçirmek — bu ünitenin `props` kavramı, tam olarak bu alışkanlığın React'teki karşılığıdır.

Ünite 1'de React'in bildirimsel modelini ve TSX söz dizimini gördünüz. Ama tek başına bir `<h1>Merhaba</h1>` döndüren bir fonksiyon henüz gerçek bir bileşen değildir — dışarıdan veri alamaz, dolayısıyla üç farklı etkinlik için üç farklı kart göstermek isteseniz bile bunu tek bir tanımla yapamazsınız. Bu ünite, bir bileşenin dışarıdan veri almasını sağlayan `props` mekanizmasını ve bunu TypeScript ile güvenli hâle getiren `interface` tanımlarını tanıtır.

4.2.1 Fonksiyon bileşeni: JSX döndüren sıradan bir fonksiyon

Fonksiyon bileşeni: JSX döndüren sıradan bir TypeScript fonksiyonu olarak yazılan bileşen (*function component*).

Bir React **Fonksiyon bileşeni**, aslında sıra dışı hiçbir şey değildir — geriye JSX döndüren, sıradan bir TypeScript fonksiyonudur. Tek katı kural, adlandırmadır: bileşen fonksiyonları büyük harfle başlamalıdır (`EtkinlikKarti`, `Koltuk` gibi). Bunun nedeni teknik bir zorunluluktur: React, küçük harfle başlayan bir adı (`etkinlikKarti` gibi) gördüğünde bunu sizin bileşeniniz değil, düz bir HTML etiketi sanır ve tarayıcıda `<etkinlikkarti>` diye anlamsız bir etiket oluşturmaya çalışır.

Ortak Hata / Tuzak

Bir bileşeni yanlışlıkla küçük harfle adlandırmak (`etkinlikKarti` gibi bir fonksiyon adı) derleme hatası vermeyebilir ama React onu bir bileşen olarak tanımaz — ekranda beklediğiniz kart yerine tuhaf, boş bir HTML etiketi görürsünüz. Adlandırma büyük harfle başlamıyorsa, önce bunu kontrol edin.

4.2.2 props: bileşene dışarıdan veri geçirmenin tek yolu

props: Bir bileşene dışarıdan geçirilen, bileşenin kendisi tarafından değiştirilmeyen salt okunur veri (*props*, "özellikler" kelimesinin kısaltması).

Bir bileşenin dışarıdan veri alabilmesinin — ör. Hangi etkinliği göstereceğini bilmesinin — tek yolu **props**tur. `props`'u bir fonksiyona parametre geçirmeye benzetebilirsiniz — ama önemli bir farkla: **salt okunur** (*read-only*) kabul edilir. Bir bileşen kendisine gelen `props`'u doğrudan değiştirmez; yalnızca okur ve bu veriye göre JSX üretir.

Analoji

`props`'u bir form dilekçesine önceden doldurulmuş bilgiler vermeye benzetin: dilekçeyi (bileşeni) dolduran kişi (üst bileşen) hangi bilgilerin (`props`) gireceğine karar verir; dilekçenin kendisi bu bilgileri değiştiremez, yalnızca olduğu gibi gösterir. Bir memur elindeki dilekçedeki adı kendiliğinden değiştirmez — yalnızca üzerinde yazanı okur.

4.2.3 `interface XProps` ile tipleme

Bir bileşenin hangi `props`'u beklediğini TypeScript'e söylemek için her bileşenin aldığı `props`'un şeklini bir `interface` ile tanımlarsınız — adlandırma kuralı, bileşen adının sonuna `Props` eklemektir (`EtkinlikKartiProps` gibi). Aşağıdaki kesit, `EtkinlikKarti` bileşeninin beklediği tek bir `etkinlik` alanını tanımlar:

```
interface EtkinlikKartiProps {
  etkinlik: Etkinlik;
}
```

Bu tanımın gücü şurada ortaya çıkar: bileşeni yanlış tipte ya da eksik bir `props` ile çağırırsanız `tsc` bunu — kod hiç çalışmadan önce, editördeyken — bir hata olarak işaretler. Ör. `<EtkinlikKarti />` yazıp `etkinlik` prop'unu hiç vermezseniz, ya da `etkinlik` alanına — `Etkinlik` yerine — bir dize geçirmeye çalışırsanız derleme başarısız olur; Hafta 3'ün `Etkinlik` arayüzünün tam olarak bu tip güvenliğini sağladığını hatırlayın.

4.2.4 `EtkinlikKarti` bileşeni

Hafta 2'nin `article.etkinlik-karti` HTML yapısını, artık bir gerçek bileşene dönüştürebilirsiniz. `EtkinlikKarti`, `etkinlik: Etkinlik` prop'unu alıp başlığı, tarihi, mekânı ve fiyatı gösterir — fiyat `0` ise ilk basit koşullu render örneğinizi görürsünüz: bir üçlü operatörle "Ücretsiz" rozeti.

```
export function EtkinlikKarti({ etkinlik }: EtkinlikKartiProps) {
  return (
    <article className="etkinlik-karti">
      <h3>{etkinlik.baslik}</h3>
      <p>
        {etkinlik.tarih} · {etkinlik.mekan}
```

```
    </p>
    <p>
      Kapasite: {etkinlik.kapasite} ·{' '}
      {etkinlik.biletFiyati === 0 ? (
        <span className="rozet-ucretsiz">Ücretsiz</span>
      ) : (
        `Bilet: ${etkinlik.biletFiyati} TL`
      )}
    </p>
  </article>
 );
}
```

`etkinlik.biletFiyati === 0 ? (...) : (...)` ifadesi, koşulun doğru olduğu durumda solundaki JSX'i, yanlış olduğu durumda sağındaki metni üretir — bu kalıba Ünite 4'te çok daha ayrıntılı döneceksiniz.

Örnek Olay 4.1: Aynı bileşen, iki farklı `props` ile iki farklı görünüm

`EtkinlikKarti` bileşenini `etkinlikler` dizisinin ilk öğesiyle (`etkinlikler[0]`, "Bahar Konseri") çağırdığınızda bileşen bu etkinliğin bilgilerini basar; aynı bileşeni dizinin ikinci öğesiyle (`etkinlikler[1]`, "Kariyer Günleri") çağırdığınızda ise ekranda "Kariyer Günleri" görünür ve bu etkinliğin `biletFiyati` alanı `0` olduğundan "Ücretsiz" rozeti otomatik belirir. Kodun kendisi tek bir yerde durur; farklı sonuç yalnızca farklı `props` geçirilmesinden kaynaklanır — "bir kez yaz, çok kez kullan" fikrinin somut hâlidir.

Analoji

Bileşeni bir yapı setinin (LEGO) tekrar kullanılabilir bir parçasına benzetin: aynı parça tasarımı, farklı renklerle (farklı `props` ile) defalarca kullanılabilir; parçanın kendisi tek bir yerde tanımlıdır, her kullanımda yeniden tasarlanmaz.

4.2.5 `children`: bileşeni sarmalayıcı olarak kullanmak

children: Bir bileşenin açılış/kapanış etiketleri arasına yazılan, o bileşene otomatik geçirilen JSX içeriği (*children*).

Bazen bir bileşene tek bir veri değeri değil, herhangi bir JSX içeriği geçirmek istersiniz. Bir bileşenin açılış ve kapanış etiketleri arasına yazılan JSX, o bileşene otomatik olarak **children** adlı özel bir `props` alanı olarak gelir. Bu, bir bileşeni "sarmalayıcı" (*wrapper*) olarak kullanmayı mümkün kılar — Etkinlik/Bilet Sisteminde bunu küçük bir `Bolum` bileşeniyle göreceksiniz: bir başlık basan, içine konan herhangi bir JSX'i altında gösteren bir sarmalayıcı.

Analoji

`children`'ı bir hediye kutusunun içine istediğiniz hediyeyi koymaya benzetin: kutunun (sarmalayıcı bileşenin) tasarımı (kurdele, kapak) sabittir; içine hangi hediyenin (içeriğin) konacağı her kullanımda değişir — kutunun kendisi hediyenin ne olduğunu bilmek/bilmemek zorunda değildir, yalnızca onu sarar.

```
interface BolumProps {
  baslik: string;
  children: ReactNode;
}
```

```
5
6  export function Bolum({ baslik, children }: BolumProps) {
7    return (
8      <section className="bolum">
9        <h2>{baslik}</h2>
10       {children}
11     </section>
12   );
13 }
```

`children: ReactNode` tipi, "herhangi bir geçerli JSX içeriği" anlamına gelir — tek bir öğe, birden fazla öğe, hatta düz metin bile olabilir. `Bolum` bileşeni içinde ne olduğunu bilmez, yalnızca aldığı `children`'ı başlığın altına yerleştirir.

> **İpucu**
>
> Bir bileşen hem sıradan `props` (ör. `baslik: string`) hem `children` alabilir — ikisi çakışmaz, aynı `interface` içinde birlikte tanımlanabilirler (`BolumProps` örneğinde olduğu gibi).

4.2.6 Bileşen ağacı ve tek yönlü veri akışı

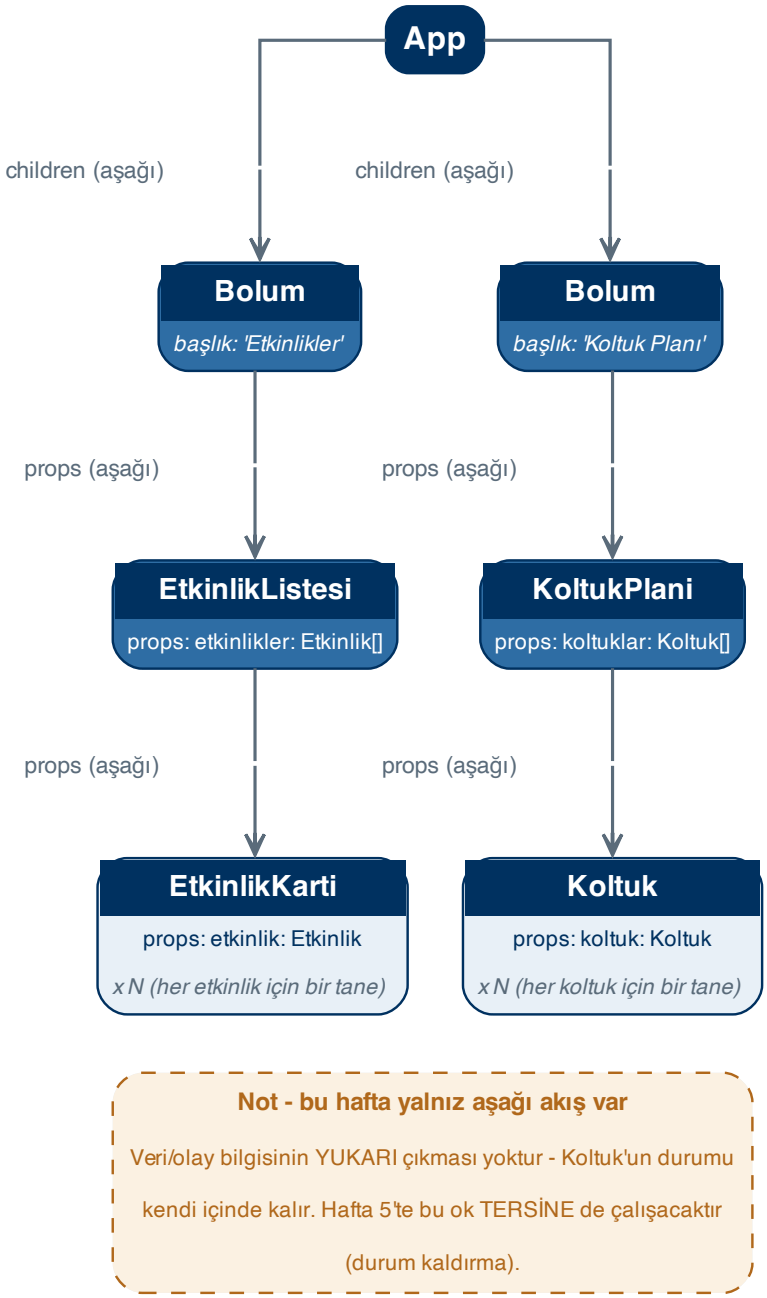
Etkinlik/Bilet Sisteminin ön yüzü artık tek bir dosya değil, birbirine bağlı bir bileşen AĞACIdır: `App` kökte, `EtkinlikListesi` ve `KoltukPlani` onun altında, `EtkinlikKarti` ve `Koltuk` en altta yer alır. Bu ağaçta `props` yalnızca yukarıdan aşağıya akar — **Tek yönlü veri akışı** bir alt bileşenin kendi verisini üst bileşene doğrudan gönderemeyeceği anlamına gelir (bu Hf.5'in "durum kaldırma" konusudur).

Tek yönlü veri akışı: props'un yalnızca üst bileşenden alt bileşene doğru akması (*one-way data flow*).

> **Analoji**
>
> Tek yönlü veri akışını bir haber bülteninin yayın zincirinde bilginin yalnızca yukarıdan aşağı aktığı bir ilan panosuna benzetin: genel merkez (üst bileşen) panoya bilgi asar, şubeler (alt bileşenler) onu okur; bir şube doğrudan panoyu değiştiremez, merkeze geri bildirim ayrı bir kanaldan gider.

Şekil 4.2 bu bileşen ağacını gösterir: kökte `App`, onun altında `EtkinlikListesi` ve `KoltukPlani`, en altta ise `EtkinlikKarti` ve `Koltuk` yer alır; `props` her okla yukarıdan aşağıya akar.

> **3 • Katılım** **Şimdi Siz Deneyin**
>
> `EtkinlikKarti` bileşenine yeni bir `vurgula: boolean` prop'u ekleyin; kartın kök öğesine `vurgula` `true` ise ek bir CSS sınıfı uygulayın. `App.tsx`'te bileşeni iki kez, birini `vurgula` değeri `true`, diğerini `false` ile çağırın ve sonucu gözlemleyin.
>
> ---
>
> **Geribildirim.** `interface EtkinlikKartiProps` içine `vurgula: boolean;` eklenir; JSX'te `className` özniteliğine bir üçlü operatörle — `vurgula` doğruysa `'etkinlik-karti vurgulu'`, değilse `'etkinlik-karti'` döndürecek şekilde — koşullu bir sınıf adı verilir. Aynı bileşen farklı `props` ile çağrıldığında farklı görünür ama kodu tek bir yerde durur — "bir kez yaz, çok kez kullan" fikri burada somutlaşır: `EtkinlikKarti` fonksiyonunu değiştirmeden, yalnızca ona geçirdi-



Şekil 4.2: Etkinlik/Bilet Sisteminin bileşen ağacı: `App` kökten `EtkinlikListesi`/`KoltukPlani` üzerinden `EtkinlikKarti`/`Koltuk`'a; `props` yalnızca yukarıdan aşağıya akar.

ğiniz veriyi değiştirerek görünümü kontrol edersiniz.

Kendinizi Sınayın 4.2

Bir `EtkinlikKarti` bileşeni `etkinlik: Etkinlik` prop'unu aldıktan sonra bileşen gövdesinde

```
etkinlik.baslik = 'Yeni Başlık';
```

şeklinde bir atama yaparsanız ne olur ve bu neden kötü bir pratiktir?

5 • İzleme — Aktarım — Başka Nerede?

`EtkinlikKarti`'nin aldığı `etkinlik: Etkinlik` prop'u, Hf.6'da FastAPI'den gelecek gerçek API verisiyle değişmeden kullanılmaya devam edecek; prop tipini burada doğru kurmak, ileride "sunucudan gelen veri hangi şekle uymalı" sorusunun cevabını şimdiden vermektedir. Aynı şekilde bugün yazdığınız `Bolum` sarmalayıcısı, Etkinlik/Bilet Sisteminin her yeni bölümünde (Hf.5'in sepet bölümü dahil) aynı başlık stiliyle yeniden kullanılacaktır.

4.3 state ve Olaylar

1 • Ön-öğretim — Neden Önemli?

`EtkinlikKarti`/`EtkinlikListesi` bileşenlerini yazdınız ama hiçbiri hiçbir şeyi hatırlamıyor — bir koltuğa tıklarsanız ne olur? Bileşenin bir "hafızası" (durumu) yoksa tıklama olayı bir hiç'tir, ekranda hiçbir şey değişmez. Bu ünite bileşenlere kendi hafızalarını kazandırır.

1 • Ön-öğretim — Ön Bilgi

Hafta 3 Ünite 1'in referans semantiğini hatırlayın: bir diziyi/nesneyi doğrudan mutasyona uğratmanın (`dizi.push(...)` gibi) dışarıdan da görünür olduğunu görmüştünüz. React'in durum güncellemesinde neden eski değeri değiştirmek yerine yeni bir değer oluşturmanız gerektiğinin temeli tam olarak budur. C/C++'ta bir `const` değişkeni değiştirememe disiplini de burada tanıdık bir köprü kurar.

Ünite 2'de bileşenlerin dışarıdan `props` aldığını gördünüz — ama `props` salt okunurdur, bir bileşenin kendi başına değişebilen bir verisi yoktu. Bir koltuğa tıklandığında rengin değişmesi için bileşenin kendi içinde saklayabileceği, tıklamayla güncelleyebileceği bir veriye ihtiyacı vardır. Bu ünite React'in bunun için sunduğu `useState` mekanizmasını, değişmezlik ilkesini ve olay işleyicilerini tanıtır.

4.3.1 `useState`: bir bileşenin kendi hafızası

state (durum): Bir bileşenin render'lar arasında hatırladığı, kendi kontrolündeki veri (*state*).

state (durum), bir bileşenin — `props` gibi dışarıdan gelen değil — kendi içinde tuttuğu ve zamanla değişebilen veridir. React bunu tutmanız için bir **Hook** sunar — `useState`, ismiyle başlayan (`use...`) özel bir React fonksiyonudur. Bir bileşenin içinde şöyle çağrılır:

```
const [durum, setDurum] = useState<KoltukDurumu>(baslangicDegeri);
```

Bu tek satır iki şey yapar: `useState` çağrısı iki elemanlı bir dizi döndürür ve siz bunu anında iki değişkene ayırırsınız (*array destructuring*) — ilk eleman (`durum`) o anki değeri, ikinci eleman (`setDurum`) bu değeri değiştirmenin tek yoludur. React, `<KoltukDurumu>` ile verilen tip parametresi sayesinde `durum`'un her zaman `KoltukDurumu` tipinde olacağını garanti eder — Hafta 3 Ünite 3'ün `Liste<T>` generic'iyle aynı fikri taşır: hangi tip için çalışacağı kullanım anında belirlenir.

Analoji

`state`'i bir ışık düğmesinin kendi konumunu (açık/kapalı) hatırlamasına benzetin: düğmeye her bastığınızda önceki konumunu unutmaz, yeni konumunu tutar; bir bileşenin `useState`'i de aynı şekilde en son değerini render'lar arasında hatırlar.

4.3.2 Değişmezlik ilkesi: eskiyi değiştirme, yenisini bildir

Değişmezlik ilkesi: Mevcut bir değeri doğrudan değiştirmek yerine her zaman yeni bir değer/nesne/dizi üretme kuralı (*immutability*).

`setDurum`'u çağırmanın tek doğru yolu, eski değeri değiştirmek değil, yeni bir değer üretip bunu `setDurum(...)`'a vermektir – buna **Değişmezlik ilkesi** denir. Hafta 3 Ünite 1'in referans semantiğini hatırlayın: bir diziyi/nesneyi doğrudan mutasyona uğratmak, o veriye işaret eden herkesi etkiliyordu. React bu davranışı state için istemez — çünkü React'in "bir şey değişti mi?" sorusunu cevaplama yöntemi **referans karşılaştırmasıdır**: eski ve yeni değerin bellekteki adresi aynıysa (siz eski nesneyi/diziyi yerinde değiştirdiyseniz), React hiçbir fark görmez ve ekranı yeniden çizmez.

Ortak Hata / Tuzak

Bir diziyi state'te tutup `durumDizisi.push(yeniEleman)` ile doğrudan değiştirmek — sonra da `setDurumDizisi(durumDizisi)` çağırmak — yaygın bir hatadır. `push` diziyi yerinde değiştirir, dizinin bellek adresi aynı kalır; `setDurumDizisi` eski adresle aynı adresi gördüğü için React hiçbir değişiklik algılamaz, ekran güncellenmez. Doğrusu `setDurumDizisi([...durumDizisi, yeniEleman])` gibi yeni bir dizi üretmektir — C++'ta bir `const` değişkeni değiştirememe disiplinine benzer, ama burada derleyici değil React'in kendisi bu kuralı koyar.

4.3.3 Olay işleyicileri: JSX'e bağlanan tepki fonksiyonları

Olay işleyicisi: onClick gibi bir JSX özniteliğine bağlanan, kullanıcı etkileşimine tepki veren fonksiyon (*event handler*).

Bir kullanıcı etkileşimine (tıklama, form gönderimi) tepki vermek için JSX özniteliklerini kullanırsınız: `onClick`, `onSubmit` gibi — bunlara **Olay işleyicisi** denir. React bunları normal tarayıcı DOM olaylarının üstüne tipli bir katman olarak sunar (ör. bir fare olayı işleyicisi `(e: React.MouseEvent) => ...` imzasını taşır) — Hafta 3'te `addEventListener('click', ...)` ile elle bağladığınız dinleyicinin doğrudan karşılığıdır, ama artık JSX'in içinde, doğrudan ilgili öğenin üzerinde tanımlanır. Bir işleyici genellikle `setX(...)` çağırarak state'i günceller — bu çağrı React'e "bu bileşeni yeniden render et" sinyalini verir.

Analoji

Olay işleyicisini bir zilin çalınca ne olacağını önceden söylemeye benzetin: kapı ziline (`onClick`) basıldığında hangi eylemin (fonksiyonun) tetikleneceği önceden tanımlanır; zil çalmadan o eylem asla çalışmaz — `tikla` fonksiyonu, `onClick` tetiklenmeden hiçbir zaman kendiliğinden çalışmaz.

4.3.4 Durumun bileşen içinde yaşaması

Bir `useState` çağrısı yalnızca tanımlandığı bileşene aittir — başka hiçbir bileşen (üst ya da kardeş) bu değeri doğrudan göremez ya da değiştiremez. Bu, bu haftanın koltuk seçiminin neden "kendi içinde" kaldığının temelidir: `Koltuk` bileşeni kendi `durum`'unu tutar, ama `KoltukPlani` ya da `App` bu değerden habersizdir.

4.3.5 `Koltuk` bileşeni: durum ve tıklama bir arada

Şimdi bu üç fikrin — `useState`, değişmezlik ilkesi, olay işleyicileri — `Koltuk` bileşeninde nasıl birleştiğini görün. `Koltuk` bileşeni bir `koltuk: Koltuk` prop'u alır (bu tip Hafta 3'ten aynen gelir) ve kendi `durum` state'ini, başlangıç değeri olarak gelen `koltuk.durum`'u kullanarak tanımlar:

```
interface KoltukProps {
  koltuk: KoltukTipi;
}
```

```
const [durum, setDurum] = useState<KoltukDurumu>(koltuk.durum);

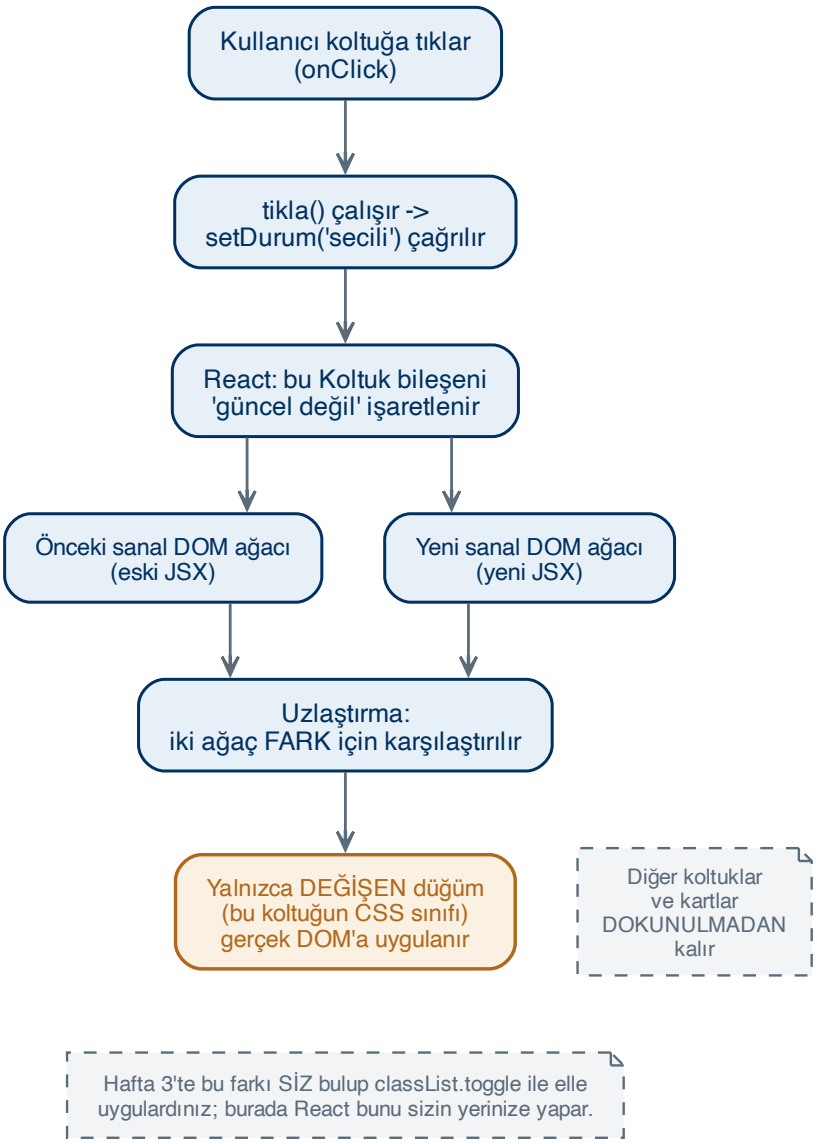
function tikla() {
  if (durum === 'dolu') return;
  setDurum((onceki) => (onceki === 'bos' ? 'secili' : 'bos'));
}
```

`tikla` fonksiyonuna dikkat edin: `durum === 'dolu'` ise erken dönüş yapar — yani `'dolu'` bir koltuğa tıklandığında hiçbir şey olmaz. Aksi halde `setDurum`'a bir fonksiyon geçirilir (`onceki => ...`); bu fonksiyon mevcut değeri (`onceki`) okuyup yeni bir değer döndürür — mutasyon yoktur, tam anlamıyla değişmezlik ilkesine uygundur. Son olarak bileşen görünümünü `durum`'a göre üretir; `disabled`/`aria-disabled` ile `'dolu'` bir koltuk fiilen tıklanamaz hâle getirilir, `&&` operatörüyle de yalnızca `'dolu'` durumunda bir "Dolu" rozeti eklenir:

```
return (
  <button
    type="button"
    className={`koltuk ${durum}`}
    onClick={tikla}
    disabled={durum === 'dolu'}
    aria-disabled={durum === 'dolu'}
  >
    {koltukEtiketi(koltuk)}
    {durum === 'dolu' && <span className="koltuk-rozet">Dolu</span>}
  </button>
);
```

> **İpucu**
>
> `disabled` özniteliği HTML düzeyinde tıklamayı zaten engeller; `aria-disabled` ise ekran okuyucular gibi erişilebilirlik araçlarına da aynı bilgiyi iletir — ikisini birlikte kullanmak iyi bir alışkanlıktır.

Şekil 4.3, `tikla` içindeki `setDurum` çağrısının tetiklediği süreci gösterir: React önce yeni bir Sanal DOM taslağı üretir, eskisiyle uzlaştırır ve yalnızca değişen `<button>` öğesini gerçek DOM'a uygular — sayfanın geri kalanı dokunulmadan kalır.



Şekil 4.3: Bir `setDurum` çağrısının tetiklediği yeniden render süreci: yeni Sanal DOM taslağı, eskisiyle uzlaştırma, yalnızca değişen düğümün gerçek DOM'a uygulanması.

3 • Katılım — Şimdi Siz Deneyin

`Koltuk` bileşeninde `durum` state'ini `useState<KoltukDurumu>(koltuk.durum)` ile tanımlayın; `tikla` fonksiyonunda `durum === 'dolu'` ise erken dönüş yapın (hiçbir şey yapmayın), aksi halde `setDurum` ile `'bos'` ↔ `'secili'` arasında geçiş yapın. Bir `'dolu'` koltuğa tıklayarak hiçbir şey olmadığını doğrulayın.

Geribildirim.

```
function tikla() {
  if (durum === 'dolu') return;
  setDurum((onceki) => (onceki === 'bos' ? 'secili' : 'bos'));
}
```

Bu güncelleme mevcut değeri okuyup yeni bir değer döndürür (mutasyon yok); `'dolu'` durumunda `tikla` erken çıkış yapar, bu yüzden `onClick` tetiklense de state değişmez, React yeniden render etmez.

Kendinizi Sınayın 4.3

```
const [durum, setDurum] = useState<KoltukDurumu>('bos');
durum = 'secili';
```

İkinci satırı yazarsanız ne olur?

5 • İzleme — Aktarım — Başka Nerede?

Bu haftaki koltuk seçimi yalnızca `Koltuk` bileşeninin kendi içinde yaşıyor — `App`'in ya da `KoltukPlani`'nın bundan haberi yok, dolayısıyla "toplam seçili koltuk sayısı" gibi bir özet şu an imkansız. Gelecek hafta bu durumu bir üst bileşene taşıyıp (durum kaldırma) birden çok bileşenin aynı bilgiyi paylaşmasını sağlayacaksınız — bu haftanın sınırı, gelecek haftanın sorusudur.

4.4 Liste ve Koşullu Render

1 • Ön-öğretim — Neden Önemli?

Etkinlik listenizde şu an sabit 3 etkinlik var, ama yarın 30 etkinlik olsaydı `EtkinlikKarti`'yi 30 kez elle mi çağırırdınız? Ve React bir listedeki hangi öğenin değiştiğini nasıl bilir — her seferinde hepsini mi yeniden çiziyor? Bu ünite bu iki soruyu — liste üretme ve `key` — yanıtlar.

1 • Ön-öğretim — Ön Bilgi

Hafta 3 Ünite 1'in `map`/`filter` dizi metotlarını hatırlayın — bir diziden yeni bir dizi üretme alışkanlığınız zaten var. Ünite 1'in Sanal DOM/uzlaştırma tanıtımını da hatırlayın — bu ünite uzlaştırmayı `key` bağlamında somutlaştırır.

Ünite 2-3'te tek bir `EtkinlikKarti`'nin ve tek bir `Koltuk`'un nasıl çalıştığını gördünüz. Ama Etkinlik/Bilet Sisteminin gerçek sayfası tek bir kart değil, bir liste gösterir — `veri.ts`'teki `etkinlikler` ve `koltuklar` dizilerinin her öğesi için bir bileşen

üretilmesi gerekir. Bu ünite, bir diziden bileşen listesi üretmeyi (`map`), her öğeye doğru kimliği vermeyi (`key`) ve boş liste gibi özel durumları yönetmeyi (koşullu render) öğretir — son olarak da tüm bu bileşenleri `App`'te birleştirip haftanın React katmanını tamamlarsınız.

4.4.1 `map` ile bileşen listesi üretme

Hafta 3'te bir `Etkinlik[]` dizisinden DOM öğeleri üretmek için `forEach` ile bir döngü kurup her turda `createElement` çağırıyordunuz — zorunlu bir yaklaşımdı. React'te bunun bildirimsel karşılığı, dizinin `.map(...)` metoduyla bir JSX öğe dizisi üretmektir: her diziden yeni bir dizi üretmek gibi, `map` de her etkinlikten bir `<EtkinlikKarti />` üretir.

4.4.2 `key` neden gerekli?

key: Bir listedeki her öğeye verilen, React'in uzlaştırmada öğe kimliğini takip etmesini sağlayan kararlı tanımlayıcı (*key*).

Bir listedeki her öğeye veriden gelen, kararlı ve benzersiz bir kimlik (`etkinlik.id`, `koltuk.id`) vermeniz gerekir — buna **key** denir. `key`, Ünite 1'de tanıttığımız uzlaştırma sürecinin "bu hangi öğeydi, hangisi değişti/eklendi/silindi" sorusunu doğru cevaplamasını sağlar. `key` olmadan — ya da dizideki sıraya (`index`) dayalı bir `key` ile — listenin başına yeni bir öğe eklendiğinde React tüm öğeleri yanlış eşleyip gereksiz yere yeniden oluşturabilir.

Analoji

`key`'i bir vestiyerdeki numaralı fişlere benzetin: her paltoya (liste öğesine) sabit bir numara (`id`) verilirse, palto sırası karışsa bile doğru palto doğru numarayla eşleşir; numara yerine yalnızca "sıradaki kanca" (`index`) kullanılırsa, araya yeni bir palto girdiğinde tüm paltolar yanlış kancalarla eşleşmiş gibi görünebilir.

Şekil 4.4, `key` olmadan (sıra/ `index` tabanlı eşleme) ve `key` ile (kararlı kimlik) bir listenin başına öğe eklendiğinde hangi öğelerin gereksiz yere yeniden oluşturulduğunu karşılaştırır.

4.4.3 `EtkinlikListesi` bileşeni: boş kontrolü ve liste

`EtkinlikListesi`, `etkinlikler: Etkinlik[]` prop'unu alır:

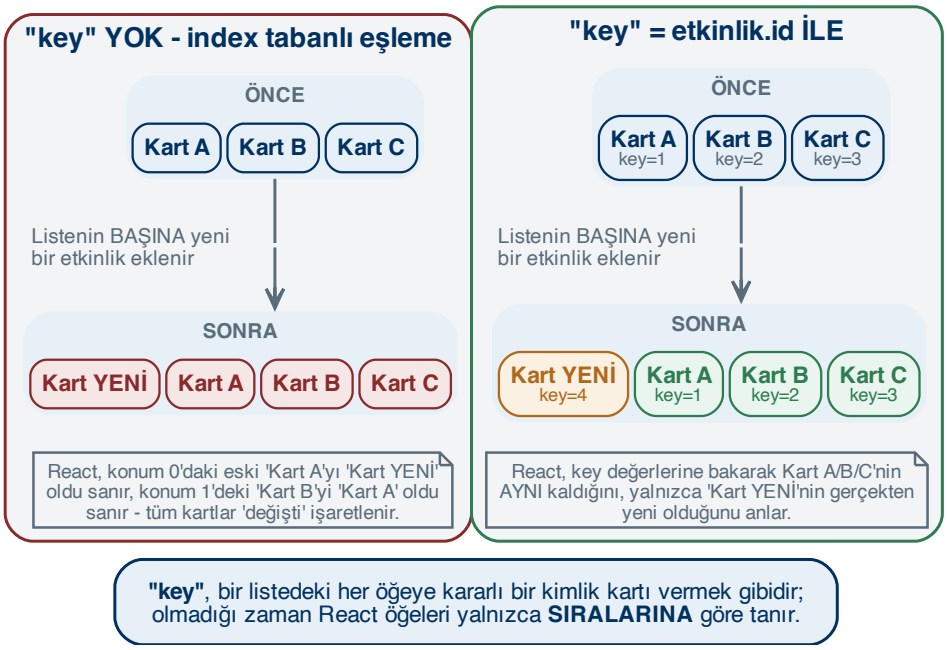
```
interface EtkinlikListesiProps {
  etkinlikler: Etkinlik[];
}
```

Koşullu render: Bir koşula göre farklı JSX (ya da hiç JSX) döndürme kalıbı (`&&`, üçlü operatör, erken dönüş) (*conditional rendering*).

Diziyi `map`'lemeden önce, özel bir durumu ele almanız gerekir: dizi boşsa ne gösterilecek? Buna **Koşullu render** denir ve üç yaygın kalıbı vardır: `&&` (bir koşul doğruysa küçük bir öğe eklemek), üçlü operatör (`? :`, iki alternatiften birini seçmek — Ünite 2'de "Ücretsiz" rozetinde görmüştünüz) ve erken dönüş (bir `if` koşuluyla erken `return` yapmak, bir listenin boş olması gibi özel bir durumu geri kalan JSX'ten önce ayrı ele almak). `EtkinlikListesi` erken dönüşü kullanır:

```
if (etkinlikler.length === 0) {
  return <p>Henüz etkinlik yok.</p>;
}
```

Dizi boş değilse, geri kalan kod çalışır ve `map` ile her etkinlik için bir `EtkinlikKarti` üretilir — her karta veriden gelen kararlı bir `key` (`etkinlik.id`) verilir:



Şekil 4.4: key olmadan (index tabanlı) ve key ile (kararlı kimlik) bir listenin başına öğe eklendiğinde uzlaştırmanın farkı.

```
return (
  <div className="etkinlik-listesi">
    {etkinlikler.map((etkinlik) => (
      <EtkinlikKarti key={etkinlik.id} etkinlik={etkinlik} />
    ))}
  </div>
);
```

Ortak Hata / Tuzak

`key` yazmayı unutmak (ya da `key` değerini her render'da `Math.random()` gibi farklı bir değer üretecek şekilde vermek) derleme hatası vermez — kod çalışır — ama React tarayıcı konsoluna bir uyarı basar ve uzlaştırma yanlış eşleme yapabilir. `key` her zaman veriden gelen, kararlı bir kimlik olmalıdır; `index` kullanmak yalnızca liste asla sıralanmıyor/değişmiyorsa güvenlidir.

4.4.4 `KoltukPlani` bileşeni: üçlü operatörle boş kontrolü

`KoltukPlani`, aynı deseni — boş kontrolü + `map` + `key` — bu kez erken dönüş yerine bir üçlü operatörle uygular:

```
interface KoltukPlaniProps {
  koltuklar: KoltukTipi[];
}
```

```
export function KoltukPlani({ koltuklar }: KoltukPlaniProps) {
  return koltuklar.length === 0 ? (
    <p>Bu etkinlik için koltuk tanımlı değil.</p>
  ) : (
    <div className="koltuk-plani">
```

```
    {koltuklar.map((koltuk) => (
      <Koltuk key={koltuk.id} koltuk={koltuk} />
    ))}
  </div>
 );
}
```

İki kalıp da (erken dönüş, üçlü operatör) aynı sonucu verir; erken dönüş, geri kalan JSX'i basitleştirmek istediğinizde (bileşenin çoğu kodu "normal" durumla ilgiliyse) daha okunaklıdır, üçlü operatör ise tüm `return` tek bir ifadede kalsın istediğinizde tercih edilir.

> **İpucu**
>
> Boş durumu hiç ele almazsanız (ör. `koltuklar.map(...)`'ı doğrudan boş bir diziye uygularsanız) kod çökmez — yalnızca boş bir `<div>` üretilir. Ama kullanıcı için bu, sayfanın bozuk mu yoksa gerçekten boş mu olduğu belirsiz kalır; bu yüzden "Bu etkinlik için koltuk tanımlı değil." gibi anlamlı bir mesaj göstermek iyi bir alışkanlıktır.

4.4.5 `App` bileşeninde birleştirme

Son adım, tüm bu bileşenleri tek bir sayfada birleştirmektir. `main.tsx`, `App` bileşenini `index.html`'in `root` öğesine bağlar:

```
createRoot(document.getElementById('root')!).render(
 <StrictMode>
  <App />
 </StrictMode>,
);
```

`App` ise Hafta 3'ün `veri.ts`'inden gelen sabit `etkinlikler`/`koltuklar` dizilerini `EtkinlikListesi` ve `KoltukPlani`'ya prop olarak geçirir — Hafta 2/Hafta 3'ün iki `<section>`'ına karşılık gelen iki bölümü, Ünite 2'nin `children` örneği olan `Bolum` bileşeniyle sarmalar:

```
export function App() {
 return (
  <>
   <Bolum baslik="Etkinlikler">
    <EtkinlikListesi etkinlikler={etkinlikler} />
   </Bolum>
   <Bolum baslik="Koltuk Planı">
    <KoltukPlani koltuklar={koltuklar} />
   </Bolum>
  </>
 );
}
```

Bu tek bileşen, bu haftanın dört ünitesinde öğrendiğiniz her şeyi bir araya getirir: TSX söz dizimi (Ünite 1), `props` ve `children` (Ünite 2), `Koltuk`'ın kendi state'i (Ünite 3, `KoltukPlani` üzerinden dolaylı olarak) ve `map`/`key`/koşullu render (bu ünite). `npm run dev` çalıştırdığınızda tarayıcıda "Etkinlikler" başlıklı bölümde 3 kart, "Koltuk Planı" başlıklı bölümde 3×4'lük bir koltuk ızgarası görürsünüz; `bos` bir koltuğa tıklandığında rengi değişir, `dolu` işaretli bir koltuk tıklanamaz.

3 • Katılım **⌨ Şimdi Siz Deneyin**

`EtkinlikListesi` bileşenindeki `etkinlikler.map(...)` çağrısındaki `key` satırını geçici olarak silin (ya da her seferinde farklı bir değer üretecek şekilde değiştirin), tarayıcı konsolunu açıp hangi uyarıyı aldığınızı okuyun; sonra `key` değerini tekrar `etkinlik.id` yapın.

Geribildirim. React konsola "her listedeki çocuğun benzersiz bir `key` prop'u olmalı" uyarısı basar; her render'da farklı bir değer üreten bir `key` kullanıldığında ise React o öğeyi her seferinde sıfırdan oluşturulmuş sanır (gereksiz yeniden oluşturma). Kararlı ve benzersiz bir `key` (burada `etkinlik.id`), React'in "bu hangi öğeydi, hangisi değişti" sorusunu doğru cevaplamasını sağlar.

Kendinizi Sınayın 4.4

```
koltuklar.map((k) => <Koltuk koltuk={k} />)
```

Yukarıdaki gibi yazıp `key` eklemeyi unutursanız kod çalışır mı, ne olur?

5 • İzleme **Aktarım — Başka Nerede?**

`EtkinlikListesi` ve `KoltukPlani` bileşenlerini `App.tsx`'te birleştirerek Etkinlik/Bilet Sisteminin ön yüzüne ilk React katmanını eklediniz. Hf.5'te bu ikisinin arasına bir köprü kuracaksınız: kullanıcının seçtiği koltukların bir sepete dönüşmesi, bugünün yerel `Koltuk` state'inin bir üst bileşene taşınmasını (durum kaldırma) gerektirecektir — bu haftanın "toplam seçili koltuk sayısını nasıl görürüz?" sorusunun cevabı orada verilecektir.

Bölüm Özeti

Özet

Bu hafta, Hafta 3'ün elle DOM güncelleyen kodunun üzerine React'i inşa ettiniz. İlk ünitede React'in bildirimsel programlama modelini — "durumu tanımlayın, görünümü React güncellesin" — Hafta 3'ün zorunlu yaklaşımıyla karşılaştırdınız, Sanal DOM ve uzlaştırma kavramlarını fikir düzeyinde gördünüz ve Vite'ın `react-ts` şablonuyla bir proje kurup TSX söz dizimini (ifade gömme, `className`, tek kök öğe/Fragment) öğrendiniz. İkinci ünitede bileşenlerin dışarıdan veri almasını sağlayan `props`'u, `interface XProps` ile tiplemeyi ve `children` ile sarmalayıcı bileşenler (`Bolum`) yazmayı gördünüz; `EtkinlikKarti` bileşenini Hafta 2'nin HTML yapısından türettiniz. Üçüncü ünitede `useState` ile bir bileşenin kendi hafızasını tutmasını, değişmezlik ilkesine (eskiyi değiştirme, yenisini bildir) uymayı ve tiplenmiş olay işleyicileriyle (`onClick`) kullanıcı etkileşimine tepki vermeyi öğrendiniz; `Koltuk` bileşeni kendi `durum`'unu taşıyan ilk örneğiniz oldu. Son ünitede `map` ile bir diziden bileşen listesi üretmeyi, her öğeye veriden gelen kararlı bir `key` vermenin uzlaştırma için neden kritik olduğunu ve koşullu render kalıplarını (`&&`, üçlü operatör, erken dönüş) öğrendiniz; `EtkinlikListesi` ve `KoltukPlani` bileşenlerini `App`'te birleştirerek Etkinlik/Bilet Sisteminin ön yüzüne ilk React katmanını eklediniz. Bu hafta bo-

yunca koltuk seçimi bilinçli olarak `Koltuk` bileşeninin kendi içinde kaldı — gelecek hafta bu durumu bir üst bileşene taşıyarak (durum kaldırma), `useEffect`, formlar ve `Context` ile bir koltuk seçme formu ve gerçek bir sepet oluşturacaksınız.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — tek kök öğe hatası). Bir bileşen **tek bir kök öğe** döndürmelidir; `<h1>` ve `<p>` doğrudan kardeş olarak döndürülemez, çünkü TSX aslında bir tek fonksiyon çağrısına derlenir ve bir fonksiyon aynı anda iki ayrı ağacı döndüremez. Çözüm ya ikisini ortak bir `<div>` ile sarmak ya da — ekstra bir DOM düğümü oluşturmak istemiyorsanız — görünmez bir Fragment (`<>...</>`) kullanmaktır; Fragment ekrana hiçbir görünür iz bırakmadan yalnızca JSX'in tek bir kök altında toplanmasını sağlar.

Soru (Ünite 2 — `props`'a doğrudan atama). TypeScript/React bu atamayı otomatik olarak engellemez — kod çalışabilir — ama `props` salt okunur kabul edilmelidir; bu, tek yönlü veri akışı ilkesinin bir parçasıdır. Bir alt bileşenin kendisine gelen veriyi doğrudan değiştirmesi, üst bileşenin haberi olmadan veriyi bozar ve React'in ne zaman yeniden render edeceğini öngörülemez hâle getirir — çünkü React değişikliği yalnızca `setX(...)` gibi kendi mekanizmaları üzerinden takip eder, doğrudan bir alan atamasını görmez. Değişiklik gerekiyorsa üst bileşene bir olay/callback ile bildirilmelidir — bu Hf.5'in "durum kaldırma" konusudur.

Soru (Ünite 3 — `state`'e doğrudan atama). `useState`'in döndürdüğü ilk değer (`durum`) `const` bağlamlıdır — doğrudan atama yapılamaz, `tsc` bu satırı bir hata olarak işaretler ve derleme başarısız olur. State'i değiştirmenin tek yolu ikinci elemanı (`setDurum`) çağırmaktır; bu çağrı React'e "bu bileşeni yeniden render et" sinyalini verir ve React yeni bir Sanal DOM taslağı üretip uzlaştırma sürecini başlatır. Doğrudan atama ise React'e hiçbir şey söylemez — React'in referans karşılaştırması hiçbir fark görmez, ekran değişmez (bu satır zaten `const` kısıtı yüzünden derlenmeyecektir bile).

Soru (Ünite 4 — `key` eksikliği). Evet, kod çalışır — derleme hatası değildir — ama React tarayıcı konsoluna bir uyarı basar. `key` olmadan React listedeki öğeleri yalnızca sıralarına (`index`) göre eşlemeye çalışır; bu da listeye yeni bir öğe eklendiğinde/sırası değiştiğinde yanlış öğenin durumunun yanlış konuma taşınmasına yol açabilir — ör. bir koltuğun `secili` durumu, aslında farklı bir koltuğa ait gibi görünebilir. Bu yüzden `key` her zaman veriden gelen, kararlı bir kimlik (`etkinlik.id`, `koltuk.id`) olmalıdır.